

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ЛАБОРАТОРНАЯ РАБОТА №2
по дисциплине «Алгоритмы и структуры данных»
Тема: «Реализация и исследование алгоритма сортировки TimSort»

Студентка гр. 4342

Киселева А.Д.

Преподаватель

Иванов Д.В.

Санкт-Петербург

2025

СОДЕРЖАНИЕ

1.	Цель работы	3
2.	Задание	4
3.	Теоретический минимум	6
4.	Выполнение работы	11
4.1.	Описание структуры кода	11
4.2.	Описание пайплайна	13
5.	Исследование	14
6.	Вывод	15
	Приложение А. Код программы	16
	Приложение Б. Тестирование	21

1. ЦЕЛЬ РАБОТЫ

Реализовать алгоритм сортировки TimSort для упорядочивания элементов целочисленного массива по убыванию модуля их значений, обеспечить его устойчивость и визуализировать процесс сортировки путем вывода промежуточных состояний массива. Исследовать асимптотическую сложность алгоритма и показать на графике зависимость времени работы от размера введенных данных.

2. ЗАДАНИЕ

Имеется массив данных для сортировки *int arr[]* размера *n*. Необходимо отсортировать его алгоритмом сортировки *TimSort* по убыванию модуля.

Так как *TimSort* - это гибридный алгоритм, содержащий в себе сортировку слиянием и сортировку вставками, то вам предстоит использовать оба этих алгоритма. Поэтому нужно выводить разделённые блоки, которые уже отсортированы сортировкой вставками.

Кратко алгоритм сортировки можно описать так:

1. Вычисление *min_run* по размеру массива *n* (для упрощения отладки *n* уменьшается, пока не станет меньше 16, а не 64)
2. Разбиение массива на частично-упорядоченные (в т.ч. и по убыванию) блоки длины не меньше *min_run*
3. Сортировка вставками каждого блока
4. Слияние каждого блока с сохранением инварианта и использованием галопа (галоп начинать после 3-х вставок подряд)

Исследование

После успешного решения задачи в рамках курса проведите исследование данной сортировки на различных размерах данных (10/1000/100000), сравнив полученные результаты с теоретической оценкой (для лучшего, среднего и худшего случаев), и разного размера *min_run*. Результаты исследования предоставьте в отчете. Для исследования используйте стандартный алгоритм вычисления *min_run* и начинайте галоп после 7-ми вставок подряд.

Примечание:

Нельзя пользоваться готовыми библиотечными функциями для сортировки, нужно сделать реализацию сортировки вручную. Сортировка должна быть устойчивой.

Формат ввода

Первая строка содержит натуральное число n - размерность массива, следующая строка содержит элементы массива через пробел.

Формат вывода

Выводятся разделённые блоки для сортировки в формате "Part i: *отсортированный разделённый массив*"

Затем для каждого слияния выводится количество вхождений в режим галопа и получившийся массив в формате "Gallops i: *число вхождений в галоп*

Merge i: *итоговый массив после слияния*"

Последняя строчка содержит финальный результат сортировки массива с надписью "Answer: "

3. ТЕОРЕТИЧЕСКИЙ МИНИМУМ

Timsort — гибридный алгоритм сортировки, сочетающий различные подходы. На массивах данных, которые содержат упорядоченные подмассивы, данный алгоритм показывает себя намного лучше других сортировок.

Основная идея алгоритма:

1. Входной массив разделяется на подмассивы фиксированной длины, вычисляемой определённым образом.
2. Каждый подмассив сортируется сортировкой вставками, сортировкой пузырьком или любой другой устойчивой сортировкой.
3. Отсортированные подмассивы объединяются в один массив с помощью модифицированной сортировки слиянием.

Введем следующие обозначения:

- n — размер входного массива.
- run — подмассив во входном массиве, который обязан быть упорядоченным одним из двух способов (для сортировки по убыванию):
 1. нестрого по убыванию $a_i \geq a_{i+1} \geq \dots$
 2. строго по возрастанию $a_i < a_{i+1} < \dots$
- $minrun$ — минимальный размер подмассива, описанного в предыдущем пункте.

Вычисление размера $minrun$

Число $minrun$ определяется на основе n . Оно не должно быть слишком большим, потому что сортировка вставками неэффективна на больших данных; также число не должно быть слишком маленьким, иначе придется

производить много итераций слияния. Оптимальная величина для n/minrun — степень двойки. Это требование обусловлено тем, что алгоритм слияния подмассивов наиболее эффективно работает на подмассивах примерно равного размера. Автором алгоритма было выбрано оптимальное значение, которое принадлежит диапазону $[32;65)$. Исключение: если $n < 64$, тогда $n = \text{minrun}$ и *Timsort* превращается в сортировку вставками. В данной работе верхняя граница *minrun* равна 16.

Берем старшие 6 бит числа n и добавляем единицу, если в оставшихся младших битах есть хотя бы один ненулевой. После таких вычислений n/minrun будет равно степени двойки или немного меньше степени двойки.

Алгоритм разбиения на подмассивы и их сортировка

Начиная с первого элемента, во входном массиве идет поиск упорядоченного подмассива *run*. Если получившийся подмассив упорядочен по возрастанию, то после вычисления *run* для текущего массива элементы переставляются так, чтобы они шли по убыванию. Если размер текущего *run* меньше *minrun*, тогда выбираются следующие за найденным подмассивом *run* элементы в количестве $\text{minrun} - \text{size}(\text{run})$. На выходе будет получен подмассив размером большим или равным *minrun*, часть которого (в лучшем случае — он весь) упорядочена. К данному подмассиву применяем сортировку вставками. Так как размер подмассива невелик и часть его уже упорядочена — сортировка работает эффективно. Переходим на следующий за подмассивом элемент. Если конец входного массива не достигнут выполняем процедуру еще раз, но уже начиная с нового элемента.

Слияние подмассивов

Перед тем, как производить слияние полученных подмассивов, определяется оптимальный порядок объединения, чтобы сохранить

стабильность алгоритма. Основная идея заключается в слиянии блоков схожего размера. Сам алгоритм можно описать следующим образом:

1. Создается стек блоков.
2. В стек добавляется следующий блок.
3. Если в стеке есть хотя бы 2 блока, то проверяются следующие инварианты:
 1. $Z > X + Y$ (проверяется, если в стеке есть хотя бы 3 блока)
 2. $Y > X$, где X , Y и Z - размеры 1-го, 2-го и 3-го блока в стеке соответственно.

Если хотя бы одно условие не выполняется, то блок Y сливается с наименьшим из X и Z (либо с X , если блока всего 2). Данный шаг выполняется, пока не будут выполнены инварианты (либо станет меньше 2-х блоков).

4. Переход к шагу 2.

Если после выполнения алгоритма в стеке остались блоки, то рассматриваются блоки X , Y и Z и блок Y сливается с наименьшим из X и Z , и так до опустошения стека.

После определения порядка слияния происходит объединение подмассивов согласно следующему алгоритму:

1. Создается результирующий массив, размер которого равен сумме размеров подмассивов.
2. Ставятся указатели текущей позиции на первые элементы обоих подмассивов.
3. На каждом шаге рассматривается значение текущих элементов в каждом подмассиве, берется большее из них, копируется в результирующий массив. Указатель текущего элемента перемещается в подмассиве, из которого был взят элемент.

4. Предыдущий пункт повторяется, пока один из подмассивов не закончится.
5. Все элементы оставшегося массива добавляются в конец результирующего массива.

Для оптимизации процесса слияния TimSort использует режим галопа. Рассмотрим пример.

Пусть первый блок A равен [1, 2, ..., 100], а второй B - [200, 201, ..., 300]. Очевидно, что после слияния итоговый массив примет вид A|B, но обычный алгоритм слияния потребует 100 операций сравнения. Поэтому если в определенный момент возникнет ситуация, что элементы берутся только из одного блока (например, если было взято 7 элементов подряд), можно сделать предположение, что данная зависимость продолжится.

Для проверки данной гипотезы и используется режим галопа. В данном примере после слияния 7 элементов из блока A слияние переходит в режим галопа, который ищет элементы после 7-го меньше текущего элемента из блока B (200). Заметим, что блоки отсортированы, поэтому вместо обычного линейного поиска можно применить модификацию бинарного поиска - неточный поиск.

Таким образом галоп будет искать в подмассиве [8, ..., 100] элемент больше 200, потратив на это максимум $\log_2(93) + 1$ операцию сравнения, то есть всего 7 сравнений. В итоге на слияние двух блоков понадобится не 100 сравнений, а 14.

Оценка сложности сортировки

Пусть k — число кусков, на которые разбился исходный массив, то есть $k = n / \text{minrun}$.

Главный факт, который понадобится для доказательства нужной оценки времени работы в $O(n * \log(n))$ — это то, что сливаемые массивы всегда имеют

примерно одинаковую длину. Можно сказать больше: пока $k > 3$ сливаемые подмассивы будут именно одинаковой длины. Безусловно, после разбиения массива на блоки длиной *minrun* последний блок может быть отличен от данного значения, но число элементов в нём не превосходит константы *minrun*.

При слиянии, длина образовавшегося слитого массива увеличивается в ≈ 2 раза. Таким образом получаем, что каждый подмассив может участвовать в не более $O(\log(n))$ операций слияния, а значит и каждый элемент будет задействован в сравнениях не более $O(\log(n))$ раз. Элементов n , откуда получаем оценку в $O(n * \log(n))$.

4. ВЫПОЛНЕНИЕ РАБОТЫ

4.1. Описание структуры кода

Для удобства работы с подмассивами и сохранения логики определения порядка сливаемых блоков реализован *class Stack*.

Поля класса:

- *int length* – поле, хранящее количество элементов стека.
- *list stack* – поле, содержащее элементы стека.

Методы класса:

- *def push(self, item: list)* — метод вставки элемента в конец стека.

Принимаемый аргумент:

item: list – вставляемый подмассив.

- *def pop(self) → list* — метод удаления элемента из конца стека.

Возвращаемое значение:

Возвращает удаленный подмассив.

- *def top(self) → list* — метод, возвращающий вершину стека.

Возвращаемое значение:

Возвращает подмассив, находящийся на вершине стека.

Алгоритм сортировки разбит на отдельные функции:

1. *def get_minrun(n: int) → int* — функция определения размера *minrun*.

Берутся старшие 4 бита числа *n* и добавляется единица, если в оставшихся младших битах есть хотя бы один ненулевой.

Принимаемые аргументы:

int n – размер введенного массива.

Возвращаемое значение:

int (n + flag) – размер *minrun*.

2. *def divide_into_runs(arr: list, minrun: int) → list* – функция деления введенного массива на подмассивы. Очередной подмассив заполняется отсортированными значениями, при надобности дополняется

оставшимися значениями для достижения размера *minrun*, упорядочивается по убыванию сортировкой вставками и добавляется в итоговый список подмассивов.

Принимаемые аргументы:

list arr – исходный массив.

int minrun – значение *minrun*.

Возвращаемое значение:

list runs – итоговый список подмассивов.

3. *def insert_sort(arr: list) → list* — функция сортировки вставками.

Принимаемые аргументы:

list arr – сортируемый массив

Возвращаемое значение:

list arr — массив, отсортированный в порядке убывания модулей.

4. *def bsearch(arr: list, target: int) → int* — функция бинарного поиска первого элемента, меньше заданного.

Принимаемые аргументы:

list arr – упорядоченный по убыванию массив.

int target – заданный элемент.

Возвращаемое значение:

int right – индекс первого элемента, который меньше заданного.

5. *def galloping_merge(arr1: list, arr2: list) → tuple* – функция объединения двух подмассивов с использованием режима галопа. Между собой сравниваются элементы двух подмассивов, больший записывается в результирующий массив. Если было 3 вставки из одного подмассива, активируется слияние галопом.

Принимаемые аргументы:

list arr1 – первый подмассив.

list arr2 – второй подмассив.

Возвращаемое значение:

tuple (rez, cnt_gallops) – кортеж, содержащий результирующий массив и количество галопов.

6. *def merge_order(blocks: list) → list* — функция, определяющая порядок слияния блоков согласно условиям инвариантов. Из нее происходит вызов функции, объединяющей подмассивы. Реализована на классе *Stack*.

Принимаемые аргументы:

list blocks – подмассивы с отсортированными по убыванию модулей значениями.

Возвращаемое значение:

list stack.pop() - полностью отсортированный массив.

7. *def TimSort(arr: list) → list* — управляющая функция, через которую вызываются другие, для получения отсортированного массива.

Принимаемые аргументы:

list arr – введенный массив.

Возвращаемое значение:

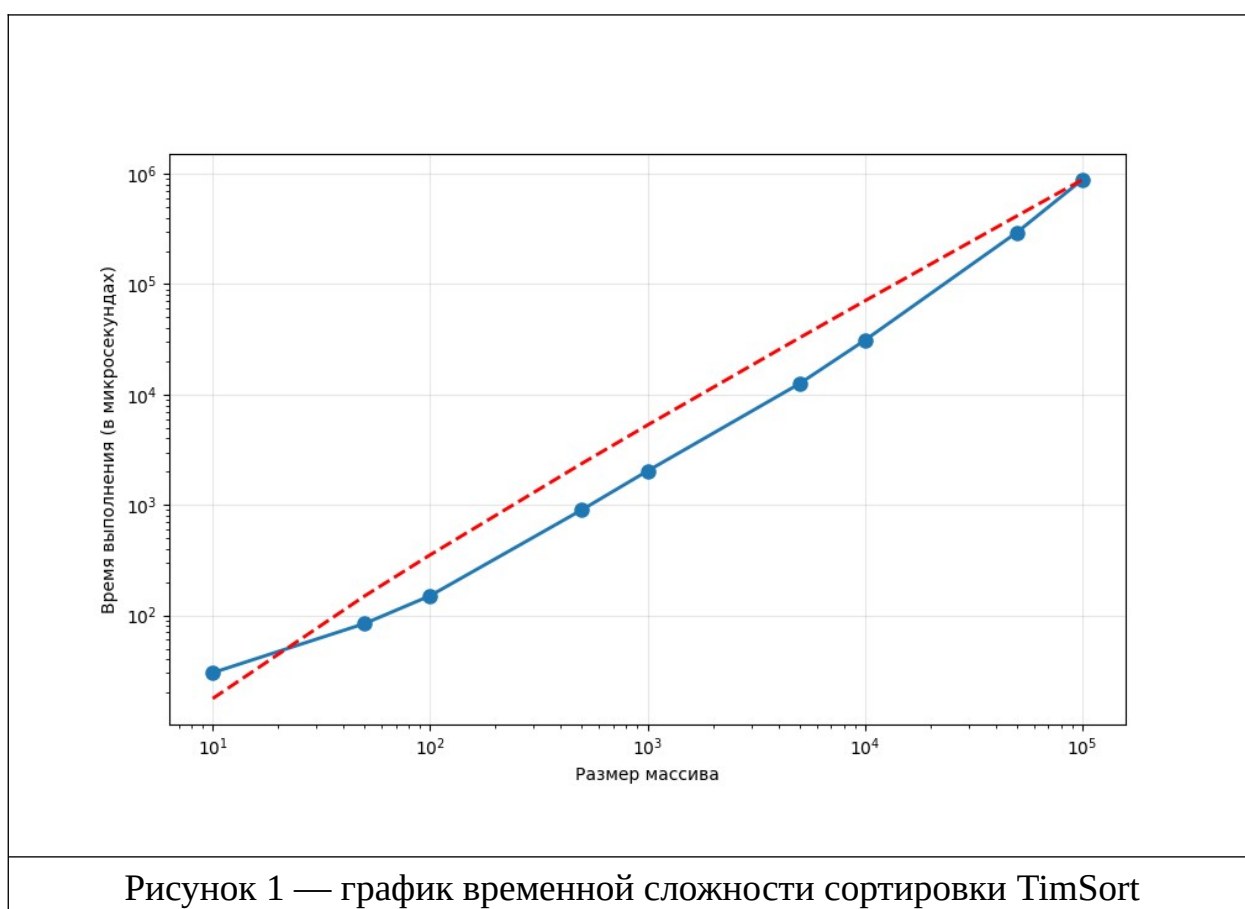
list res – отсортированный по убыванию модулей массив.

4.2. Описание пайплайна

Считывание данных происходит из потока *stdin* через функцию *input()*. Размер массива приводится к типу *int*, значения массива с помощью функции *map* разделяются, приводятся к *int* и помещаются в список. Работа с размером массива происходит в функции *get_minrun*. Работа с массивом начинается в функции *TimSort*, где вызывается *divide_into_runs* для его разбиения и *insert_sort* для сортировки подмассивов. Затем переходит в *merge_order*, оттуда направляется в *galloping_merge* и *bsearch*. Подмассивы собираются в *merge_order*, итоговый отсортированный массив возвращается в *TimSort*. Вывод данных производится за пределами функций с помощью *print()*.

ИССЛЕДОВАНИЕ

Для оценки временной сложности алгоритма сортировки TimSort было проведено исследование его работы на различных объемах данных (от 10 до 100000 случайных элементов в диапазоне от -1000 до 1000). При построении графика зависимости времени сортировки от количества элементов (см. рис. 1) полученные значения оказались ниже теоретического графика $n \cdot \log n$. Это значит, что на практике сложность алгоритма TimSort составляет $O(n \cdot \log n)$.



6. ВЫВОД

В результате выполнения лабораторной работы изучен и реализован алгоритм сортировки *TimSort*, упорядочивающий элементы целочисленного массива по убыванию модуля их значений. Обеспечена устойчивость алгоритма, а также визуализирован процесс сортировки путем вывода промежуточных состояний массива. Создан дополнительный класс *Stack* для удобства работы с данными. Исследована асимптотическая сложность алгоритма, построен график, отражающий зависимость времени работы сортировки от размера исходного массива.

ПРИЛОЖЕНИЕ А

КОД ПРОГРАММЫ

```
class Stack:
    def __init__(self):
        self.length = 0
        self.stack = []

    def push(self, item):
        self.stack.append(item)
        self.length += 1

    def pop(self):
        if not self.length:
            return 0
        removed = self.stack.pop()
        self.length -= 1
        return removed

    def top(self):
        return self.stack[-1]

def get_minrun(n: int) -> int:
    flag = 0
    while n >= 16:
        flag |= n & 1
        n >>= 1
    return n + flag

def divide_into_runs(arr: list, minrun: int) -> list:
    runs = []
    cur_run = []
    n = len(arr)
    sort_flag = 0
    ind_run = 0
    i = 0

    while i < n:
        while i < n - 1 and abs(arr[i]) < abs(arr[i + 1]) and
sort_flag != -1:
            cur_run.append(arr[i])
            sort_flag = 1
            i += 1
        while i < n - 1 and abs(arr[i]) >= abs(arr[i + 1]) and
sort_flag != 1:
            cur_run.append(arr[i])
            sort_flag = -1
            i += 1
        cur_run.append(arr[i])

        if sort_flag == 1:
            cur_run.reverse()
```



```

        while i < n - 1 and len(cur_run) < minrun:
            i += 1
            cur_run.append(arr[i])

        cur_run = insert_sort(cur_run)
        runs.append(cur_run)
        print(f"Part {ind_run}:", *cur_run)
        i += 1
        ind_run += 1
        cur_run = []
        sort_flag = 0

    return runs

def insert_sort(arr: list) -> list:
    arr_length = len(arr)
    i = 1
    while i < arr_length:
        j = i
        while j > 0 and abs(arr[j]) > abs(arr[j - 1]):
            cur = arr[j - 1]
            arr[j - 1] = arr[j]
            arr[j] = cur
            j -= 1
        i += 1
    return arr

def bsearch(arr: list, target: int) -> int:
    left = 0
    right = len(arr)
    while left < right:
        mid = (left + right) // 2
        if abs(arr[mid]) < abs(target):
            right = mid
        else:
            left = mid + 1
    return right

def galloping_merge(arr1: list, arr2: list):
    rez = []
    first_cnt = 0
    second_cnt = 0
    first_ind = 0
    second_ind = 0
    cnt_gallops = 0

    while first_ind < len(arr1) and second_ind < len(arr2):
        if abs(arr1[first_ind]) >= abs(arr2[second_ind]):
            rez.append(arr1[first_ind])
            first_ind += 1
            first_cnt += 1
            second_cnt = 0
        else:
            rez.append(arr2[second_ind])
            second_ind += 1

```

```

        second_cnt += 1
        first_cnt = 0

        if first_cnt >= 3 and second_ind < len(arr2):
            found_ind = bsearch(arr1[first_ind:], arr2[second_ind]) +
first_ind
            rez.extend(arr1[first_ind:found_ind])
            first_ind = found_ind
            first_cnt = 0
            cnt_gallops += 1

        if second_cnt >= 3 and first_ind < len(arr1):
            found_ind = bsearch(arr2[second_ind:], arr1[first_ind]) +
second_ind
            rez.extend(arr2[second_ind:found_ind])
            second_ind = found_ind
            second_cnt = 0
            cnt_gallops += 1

        rez.extend(arr1[first_ind:])
        rez.extend(arr2[second_ind:])

        return rez, cnt_gallops

def merge_order(blocks: list) -> list:
    stack = Stack()
    cnt_merges = 0

    for block in blocks:
        stack.push(block)
        while stack.length >= 2:
            if stack.length == 2:
                X = stack.pop()
                Y = stack.pop()
                if len(Y) <= len(X):
                    merged_blocks, cnt_gallops = galloping_merge(Y, X)
                    stack.push(merged_blocks)
                    print(f"Gallops {cnt_merges}:", cnt_gallops)
                    print(f"Merge {cnt_merges}:", *stack.top())
                    cnt_merges += 1
                else:
                    stack.push(Y)
                    stack.push(X)
                    break

            elif stack.length >= 3:
                X = stack.pop()
                Y = stack.pop()
                Z = stack.pop()
                if len(Z) <= len(Y) + len(X) or len(Y) <= len(X):
                    if len(Z) > len(X):
                        stack.push(Z)
merged_blocks, cnt_gallops =
galloping_merge(Y, X)
                    stack.push(merged_blocks)
                    print(f"Gallops {cnt_merges}:", cnt_gallops)
                    print(f"Merge {cnt_merges}:", *stack.top())

```

```

        else:
            merged_blocks, cnt_gallops =
galloping_merge(Y, Z)
            stack.push(merged_blocks)
            print(f"Gallops {cnt_merges}:", cnt_gallops)
            print(f"Merge {cnt_merges}:", *stack.top())
            stack.push(X)

            cnt_merges += 1

    else:
        stack.push(Z)
        stack.push(Y)
        stack.push(X)
        break

while stack.length != 1:
    if stack.length == 2:
        X = stack.pop()
        Y = stack.pop()
        merged_blocks, cnt_gallops = galloping_merge(Y, X)
        stack.push(merged_blocks)
        print(f"Gallops {cnt_merges}:", cnt_gallops)
        print(f"Merge {cnt_merges}:", *stack.top())
        cnt_merges += 1

    elif stack.length >= 3:
        X = stack.pop()
        Y = stack.pop()
        Z = stack.pop()

        if len(Z) > len(X):
            stack.push(Z)
            merged_blocks, cnt_gallops = galloping_merge(Y, X)
            stack.push(merged_blocks)
            print(f"Gallops {cnt_merges}:", cnt_gallops)
            print(f"Merge {cnt_merges}:", *stack.top())

        else:
            merged_blocks, cnt_gallops = galloping_merge(Y, Z)
            stack.push(merged_blocks)
            print(f"Gallops {cnt_merges}:", cnt_gallops)
            print(f"Merge {cnt_merges}:", *stack.top())
            stack.push(X)

        cnt_merges += 1

    return stack.pop()

def TimSort(arr: list) -> list:
    minrun = get_minrun(len(arr))
    runs = divide_into_runs(arr, minrun)
    res = merge_order(runs)
    return res

n = int(input())
arr = list(map(int, input().split()))

```

```
ans = TimSort(arr)
print("Answer:", *ans)
```

ПРИЛОЖЕНИЕ Б

ТЕСТИРОВАНИЕ

```
import pytest
from main import *

def test_decreasing_arr():
    arr = [20, -19, -18, -17, 16, 15, 13, -12, 10, -8, 7, 5, -4, 3, 2, 1]
    assert TimSort(arr) == [20, -19, -18, -17, 16, 15, 13, -12, 10, -8, 7, 5, -4, 3, 2, 1]

def test_increasing_arr():
    arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 14, 15, 16, 17, 18]
    assert TimSort(arr) == [18, 17, 16, 15, 14, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]

def test_stability():
    arr = [-1, 1, -1, 1, 2, 0, -10, -7, -7, 8, 6, 6, -6, 6, -6, -6, -6]
    assert TimSort(arr) == [-10, 8, -7, -7, 6, 6, -6, 6, -6, -6, -6, 2, -1, 1, -1, 1, 0]

def test_huge_arr():
    arr = [5, 5, 0, -4, 7, 2, -3, -4, 0, 5, 7, 8, 2, 7, -4, 4, 7, 0, 4, -2, 3, -2, -6, 6, 0, 8, -3, 4, -4, -6, 8, -4, -3, -4, 2]
    assert TimSort(arr) == [8, 8, 8, 7, 7, 7, 7, -6, 6, -6, 5, 5, 5, -4, -4, -4, 4, 4, 4, -4, -4, -4, -3, 3, -3, -3, 2, 2, -2, -2, 2, 0, 0, 0, 0]

def test_single_arr():
    arr = [11]
    assert TimSort(arr) == [11]

def test_parts():
    arr = [1, -2, 3, -4, 5, 6, -7, -8, 9, -10, 11, -10, -9, 8, 7, -7, -6, 6, 5, 4]
    parts = divide_into_runs(arr, get_minrun(len(arr)))
    assert len(parts) == 2
    assert len(parts[0]) == 11
    assert len(parts[1]) == 9

def test_gallop_merge():
    arr1 = [20, 19, 18, 1, 0]
    arr2 = [21, 17, 16, 2, -2, -1]
    expect_arr, cnt_gallops = galloping_merge(arr1, arr2)
    assert expect_arr == [21, 20, 19, 18, 17, 16, 2, -2, -1, 1, 0]
    assert cnt_gallops == 2
```